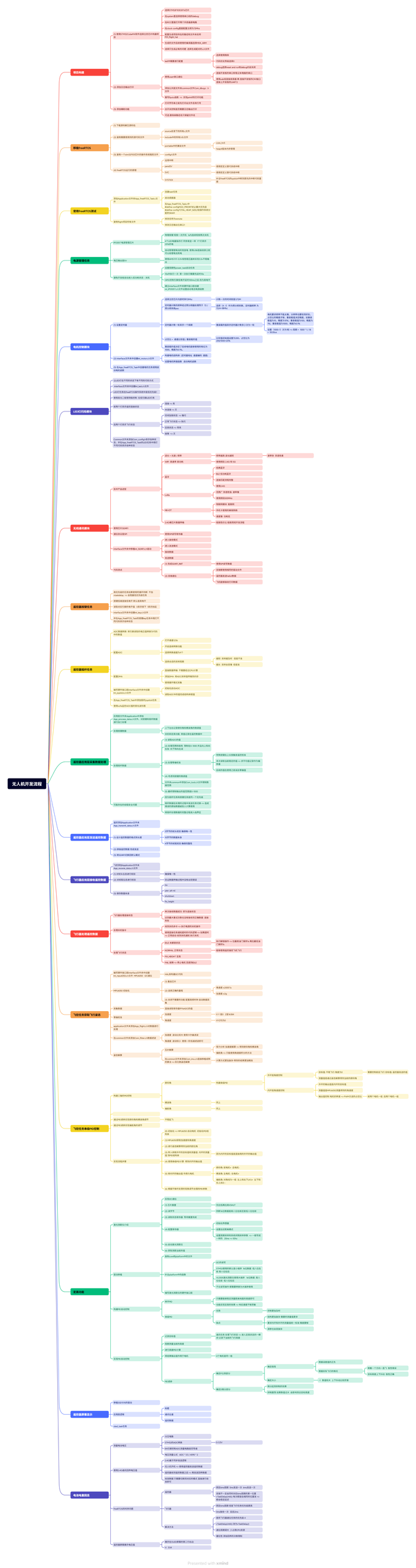

无人机开发流程

- **项目构建**
  - (1) 使用STM32CubeMX软件选择主控芯片构建项目
    - 选择STM32F103C8T6芯片
    - 在system里选择使用串口线的debug
    - 在RCC里面打开两个外部晶振电路
    - 在clock config里面配置主频为72Mhz
    - 配置生成项目所在的路径和文件夹名称P01_flight_hal
    - 生成的文件后续使用的编译器选择MDK_ARM
    - 选择只生成必有的问题 选择生成配对的.c.h文件
    - keil中需要进行配置
      - 选择使用微库
      - 代码优化等级选择0
      - debug选择reset and run和debug内容关闭
  - (2) 添加日志输出打印
    - 使用usart串口通信
      - 连接开发板的串口和笔记本电脑的串口
      - 使用usb线连接烧录器 再 连接开发板的CN1接口 连接上开发板的UART2
    - 添加公共层文件夹common文件Com_dbug.c .h文件
    - 重写fputc函数 => 实现printf的打印功能
  - (3) 添加辅助功能
    - 打印字符串之前先打印出文件名和行号
    - 总开关控制是否需要日志输出打印
    - 可选 删除掉路径名只保留文件名
- **移植freeRTOS**
  - (1) 下载源码解压源码包
  - (2) 复制需要使用到的源代码文件
    - source目录下的所有.c文件
    - include中的所有.h头文件
    - portable中的兼容文件
      - core_tm3
      - heap4版本内存管理
  - (3) 复制一个stm32f103芯片的操作系统裁剪文件
    - config.h文件
  - (4) freeRTOS运行的原理
    - 运用中断
    - pendSV
      - 使用宏定义替代系统中断
    - SVC
      - 使用宏定义替代系统中断
    - SYSTICK
      - 补全freeRTOS的systick中断到原先的中断代码里面
- **使用freeRTOS测试**
  - 添加Application文件夹App_freeRTOS_Task.c文件
    - 创建task任务
    - 启动调度器
    - 在App_freeRTOS_Task.c中 #define configMAX_PRIORITIES//最大优先级 #define configTOTAL_HEAP_SIZE//给操作系统分配的SRAM
  - 复制flight项目所有文件
    - 修改名称为remote
    - 修改日志输出位串口1
- **电源管理任务**
  - IP5305T电源管理芯片
    - 物理按键 短按一次开机 1s内连续短按两次关机
    - 4个LED电量指示灯 同充电宝一样 1个灯表示25%的电
    - 自动管理锂电池的充放电 使用USB连接烧录口就可以给锂电池充电
  - 电压输出是5V
    - 使用AMS1117-3.3V线性稳压器来实现3.3v平稳输出
  - 避免开发板自动进入低功耗状态 - 关机
    - 设置周期性power_task启动任务
    - 10s中执行一次 第一次执行需要先延时10s
    - GPIO控制引脚低电平延时100ms之后 改为高电平
    - 建立interface文件夹硬件接口层创建int_IP5305T.c.h文件设置启动电池电源函数
- **电机控制模块**
  - (1) 设置定时器
    - 选择主控芯片内部时钟72Mhz
      - 计数一次的时间就是1/72M
    - 定时器计数的频率经过预分频器处理等于 72 / 预分频系数psc
      - 选择（4 -1）作为预分频系数，定时器频率 为72/4=18Mhz
    - 定时器计数一轮表示一个周期
      - 重装载的值表示定时器计数多少次为一轮
        - 电机要求频率不能太慢，分辨率也要恰到好处，占空比的精度才够，重装载值决定精度。如重装载值为10，精度为10%；重装载值为100，精度为1%；重装载值为1000，精度为0.1%
        - 设置（1000-1）次为1轮 => 周期 = 1000 * 1 / 18 = 55.55us
    - 占空比 = 通道比较值 / 重装载的值
      - 比较值初始值设置为200，占空比为200/1000=20%
    - 重装载的值决定了后续电机能够使用的档位为1000，精度为0.1%。
  - (2) interface文件夹中创建Int_motor.c.h文件
    - 构建电机结构体（定时器地址 通道编码 速度）
    - 设置电机转速函数 启动电机函数
  - (3) 在App_freeRTOS_Task中创建电机任务调用启动电机函数
- **LED灯闪烁模块**
  - 让LED灯在不同的状态下有不同的闪烁方式
  - interface文件夹中创建Int_led.c.h文件
  - LED灯任务在freeRTOS操作系统中是低优先级1
  - 使用发光二极管阴极控制 拉低引脚LED灯亮
  - 前两个灯表示遥控连接状态
    - 连接 => 亮
    - 未连接 => 灭
  - 后两个灯表示飞行状态
    - 空闲加锁状态 => 慢闪
    - 正常飞行状态 => 快闪
    - 定高状态 => 常亮
    - 故障 => 灭
  - Common文件夹添加Com_config.h表示各种状态，并在App_freeRTOS_Task的LED任务中用灯不同闪烁表示各种状态
- **无线通讯模块**
  - 技术产品选型
    - 波长 = 光速 / 频率
      - 频率越高 波长越短
        - 速率快 穿透性差
    - WIFI 高速率 高功耗
      - 使用频段 2.4G 和 5G
    - 蓝牙
      - 经典蓝牙
      - BLE 低功耗蓝牙
      - 连接匹配流程完整
      - 使用2.4G
    - LoRa
      - 范围广 穿透性强 速率慢
      - 使用频段500MHz
    - NB-IOT
      - 物联网模块 能联网
      - 手机卡使用的蜂窝网络
      - 速度慢 功耗低
    - 2.4G裸芯片数据传输
      - 极致性价比 极致简短开发流程
  - 使用芯片SI24R1
  - 通信协议是SPI
    - 使用SPI读写寄存器
  - interface文件夹中移植Int_SI24R1.c.h驱动
    - 进入接收模式
    - 进入发送模式
    - 接收数据
    - 发送数据
  - 代码测试
    - (1) 完成SI24R1_INIT
      - 使用SPI读写数据
    - (2) 双端通信
      - 双端都使用相同的驱动文件
      - 遥控器发送hello!数据
      - 飞机能够接收打印数据
- **遥控器按键任务**
  - 高优先级的任务如果使用死循环判断 不加vtaskdelay => 会阻塞低优先级任务
  - 按键低端连接低电平 默认是高电平
  - 读取对应引脚的电平值 0表示按下 1表示抬起
  - interface文件夹中创建Int_key.c.h文件
  - 并在App_freeRTOS_Task的按键key任务中用打不同闪烁表示各种状态
- **遥控器摇杆任务**
  - ADC数据转换 将引脚读取的电压值转换为代码中的数值
  - 配置ADC
    - 打开通道1236
    - 开启连续转换功能
    - 选择转换通道为4个
    - 选择合适的采样周期
      - 越短 采样越及时 但是不准
      - 越长 采样会变慢 但是准
  - 配置DMA
    - 直接数据传输 不需要经过CPU计算
    - 添加DMA 将ADC采样值传输到内存
    - 使用循环模式采集
  - 编写硬件接口层interface文件夹中创建Int_joystick.c.h文件
    - 初始化启动ADC
    - 读取ADC中的值完成结构体赋值
  - 在App_freeRTOS_Task中添加摇杆joystick任务
  - 使用vofa监控ADC值的变化波形图
- **遥控器应用层采集数据处理**
  - 应用层文件夹Application中添加App_process_data.c.h文件。对按键和摇杆数据进行加工处理
  - 处理按键数据
    - 上下左右记录俯仰角和横滚角的微调值
    - 关机和定高功能 将值记录在遥控数据中
  - 处理摇杆数据
    - (1) 读取ADC的值
    - (2) 处理范围和极性 限制在0-1000 并且向上和向右加 向下和向左减
    - (3) 处理零偏校准
      - 特殊按键右上长按触发遥控校准
      - 多次读取当前稳定的值 => 求平均值记录作为偏移量
      - 后续的值在使用之前减去零偏值
    - (4) 考虑到按键的微调值
    - 文件夹common中添加Com_tool.c.h文件限制数据范围
    - (5) 最终限制输出的值范围是0-1000
  - 可能存在的线程安全问题
    - 因为摇杆任务和按键任务是同一个优先级
    - 摇杆数据在处理的过程中发送任务切换 => 造成错误的原始数据被别人计算使用
    - 将摇杆处理数据的完整过程进入临界区
- **遥控器应用层发送遥控数据**
  - 遥控添加Application文件夹App_transmit_data.c.h文件
  - (1) 设计遥控数据的格式和长度
    - 3字节的帧头校验 确保唯一性
    - 10字节的数据本身
    - 4字节的帧尾校验 确保完整性
  - (2) 拼接遥控数据 完成发送
  - (3) 将SI24R1切换回默认模式
- **飞行器应用层接收遥控数据**
  - 飞控添加Application文件夹App_recevie_data.c.h文件
  - (1) 对帧头信息进行校验
    - 确保唯一性
  - (2) 对帧尾信息进行校验
    - 验证数据传输过程中没有出现错误
  - (3) 保存数据本身
    - thr
    - yaw pit rol
    - shutdown
    - fix_height
- **飞行器处理遥控数据**
  - 飞行器处理连接状态
    - 单次接收数据成功 即为连接状态
    - 达到最大重试次数也没有接收到正确数据 连接失败
  - 处理关机指令
    - 收到关机命令 => 执行电源的关机操作
    - 使用直接任务通知进行的代码逻辑 => 如果超时 => 正常启动 收到关机通知 执行关机
  - 处理飞行状态
    - IDLE 未解锁状态
      - 执行解锁操作 => 拉最高油门保持1s 再拉最低油门保持1s
    - NORMAL 正常状态
      - 能够使用遥控操控飞机飞行
    - FIX_HEIGHT 定高
    - FAIL 故障 => 停止电机 回退到IDLE
- **飞控任务获取飞行姿态**
  - 编写硬件接口层interface文件夹中创建Int_mpu6050.c.h文件 MPU6050 I2C通信
    - HAL库构建I2C代码
  - MPU6050 初始化
    - (1) 重启芯片
    - (2) 选择正确的量程
      - 角速度 ±2000°/s
      - 加速度 ±2g
    - (3) 关闭不需要的功能 配置高频时钟 启动数据采集
  - 采集数据
    - 直接读取寄存器中16ADC的值
  - 零偏校准
    - 加速度
      - X Y 是0 Z是16384
    - 角速度
      - XYZ均为0
  - application文件夹添加App_flight.c.h对数据进行处理
  - 在common文件夹添加Com_filter.c.h数据滤波
    - 加速度 波动比较大 使用卡尔曼滤波
    - 角速度 波动较小 使用一阶低通滤波即可
  - 姿态解算
    - 互补解算
      - 受力分析 加速度解算 => 得到俯仰角和横滚角
      - 偏航角 => 只能使用角速度积分的方法
    - 在common文件夹添加Com_imu.c.h直接移植成熟的算法 => 四元数姿态解算
      - 计算方式更加复杂 得到的结果更加精准
- **飞控任务串级PID控制**
  - 构建三轴的PID控制
    - 俯仰角
      - 构建串级PID
        - 外环是角度控制
          - 目标值: 平稳飞行 角度为0
            - 需要控制前后飞行 目标值: 遥控器发送的值
          - 测量值是通过姿态解算得到当前的俯仰角
        - 内环是角速度控制
          - 外环的输出值是内环的目标值
          - 测量值是MPU6050测量得到的角速度
          - 输出值控制 电机的转速 => PWM方波的占空比
            - 前两个电机一组 后两个电机一组
    - 横滚角
      - 同上
    - 偏航角
      - 同上
  - 通过PID调参实现俯仰角和横滚角调节
    - 平稳起飞
  - 通过PID调参实现偏航角的调节
  - 实现流程步骤
    - (0) 初始化 => MPU6050 启动电机 初始化PID结构体
    - (1) MPU6050获取加速度和角速度
    - (2) 进行姿态解算得到当前的欧拉角
    - (3) 传入参数外环的目标值和测量值 内环的测量值 到PID结构体
      - 因为内环的目标值是直接用的外环的输出值
    - (4) 使用串级PID计算 得到内环的输出值
    - (5) 将内环的输出值 作用与电机
      - 俯仰角: 前电机+ 后电机-
      - 横滚角: 左电机- 右电机+
      - 偏航角: 对角线为一组 左上和右下(A1)+ 左下和右上(B2) -
    - (6) 根据平衡杆反馈的现象调节合理的PID参数
- **定高功能**
  - 激光测距仪介绍
    - 实现I2C通信
    - (1) 芯片重置
      - 先拉低再拉高XSHUT
    - (2) 读字节
      - 判断16位数据是高八位在前还是低八位在前
    - (3) 读取状态寄存器 等待重置完成
    - (4) 配置寄存器
      - 初始化传感器
      - 设置远近距离模式
      - 设置周期采样和系统间隔采样参数 => 一般写成一样的 20ms => 50hz
    - (5) 启动激光测距仪
    - (6) 获取测距当前的值
  - 驱动移植
    - 复制core和platform中的文件
    - 补全platform中的函数
      - I2C的读写
      - STM32使用的默认是小端序 16位数据 低八位在前 高八位在后
      - VL5305激光测距仪使用大端序 16位数据 高八位在前 低八位在后
      - 不论读写操作 都需要转换为大端序使用
    - 编写激光测距仪的硬件接口层
  - 构建PID自动控制
    - 单环PID
      - 只需要能够稳定测量距离地面的高度即可
      - 也能实现定高的效果 => 响应速度不够灵敏
    - 串级PID
      - 优势
        - 控制更加及时
      - 缺点
        - 结构更加复杂 需要的测量值更多
        - 要求内环和外环的测量值统一标准 精度要够
        - 调参也会变复杂
  - 实现PID自动控制
    - 记录目标值
      - 通讯任务 处理飞行状态 => 进入定高状态的一瞬间 记录下当前的飞行高度
    - 周期测量当前的高度
    - 进行高度PID计算
    - 将结果输出值作用于电机
      - 4个电机是同一组
    - PID调参
      - 确定P比例部分
        - 确定极性
          - 根据误差值的正负
          - 根据实际飞行的情况
            - 朝着一个方向一直飞 极性错误
            - 目标高度上下抖动 极性正确
        - 确定大小
          - -1 数值较大 上下抖动比较厉害
      - 确定D微分部分
        - 微分起到抑制的效果
        - 抑制震荡 如果数值过大 会影响到达目标高度
- **遥控器屏幕显示**
  - 移植SSD1315的驱动
  - 应用层逻辑
    - 标题
    - 通讯信道
    - 遥控数据
  - oled_task任务
- **电池电量回显**
  - 测量电池电压
    - 分压电路
    - STM32的ADC转换
      - 0-3.3V
    - EN引脚控制ADC测量电路是否导通
    - 电压测量公式 ADC * 3.3 / 4095 * 2
  - 使用2.4G通讯回传电压值
    - 2.4G属于同步发送逻辑
    - 无人机开机 => 使用遥控器发送遥控数据
    - 遥控器收到遥控数据之后 => 再发送回传数据
    - 收发数据 只需要切换到对应的模式 直接进行收发即可
  - freeRTOS的时序问题
    - 遥控器
      - 固定6ms周期 0ms发送一次 6ms发送一次
      - 双端不一定会同时对应6ms周期的某一位置 vTaskDelayUntil() 每次都是在相同的位置发 => 都会稳定延迟
    - 飞行器
      - 固定6ms周期 但是飞行任务优先级更高
      - 0ms接收一次 延后2ms
    - 解决方法
      - 提供飞行器通信任务的优先级 4
      - vTaskDelayUntil() 改为vTaskDelay()
      - 通信周期调大 少占用CPU资源
      - 健壮性 添加回传的次数限制
  - 遥控器屏幕展示电压值
    - 展示在OLED屏幕的第二行右边
    - V: 3.54

Presented with xmind